

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 6

End-to-End Study of RNN, LSTM and GRU for Sequence Learning and Video Understanding

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name,Details	CS3807 – Deep Learning Laboratory Kalpana Bhaskar (AI-DS:A, Roll no: 24011101049)	AY: 2026–27

1. Objective

The objective of this experiment is to develop an end-to-end understanding of recurrent sequence learning by implementing and comparing Vanilla RNN, LSTM and GRU models. The experiment also introduces Backpropagation Through Time (BPTT), the limitations of conventional RNNs, and the use of CNN-extracted features with recurrent networks for video understanding.

The experiment uses the UCI Human Activity Recognition Using Smartphones dataset as the primary sequence dataset. Students will prepare temporal input sequences, visualize them, train RNN/LSTM/GRU models, compare their performance, analyze convergence and generalization, and extend the pipeline to video understanding using CNN feature extraction followed by an LSTM or GRU.

A small synthetic sequence-to-sequence task is included at the end to demonstrate the encoder–decoder framework.

2. Learning Outcomes

After completing this experiment, students will be able to:

- represent sequential data using the (N, T, F) input format;
- explain the architecture and operation of a Vanilla RNN;
- explain Backpropagation Through Time and the vanishing/exploding gradient challenges;
- implement sequence classification using SimpleRNN, LSTM and GRU;
- compare RNN, LSTM and GRU using quantitative performance measures;
- interpret training/validation curves and confusion matrices;
- explain the role of CNNs in extracting spatial features from video frames;
- construct a CNN–LSTM or CNN–GRU pipeline for video understanding;
- explain the encoder–decoder architecture for sequence-to-sequence learning; and
- draw conclusions using predictive performance, model complexity and computational cost.

3. Dataset and Experimental Setup

Primary Dataset: UCI Human Activity Recognition Using Smartphones

The UCI Human Activity Recognition (HAR) dataset contains smartphone sensor measurements collected from subjects performing six activities:

- WALKING
- WALKING_UPSTAIRS
- WALKING_DOWNSTAIRS
- SITTING
- STANDING
- LAYING

The experiment should use the raw inertial signal files rather than only the already-computed 561-feature vectors. Each activity window contains 128 temporal measurements. The three-axis body acceleration, three-axis gyroscope and three-axis total acceleration signals provide nine channels.

Thus, the desired input representation is

$$X \in \mathbb{R}^{N \times 128 \times 9}.$$

The dataset can be obtained from:

<https://archive.ics.uci.edu/dataset/240/human+activity+recognition+using+smartphones>

Recommended laboratory subset: To keep execution time small, use the first 1500–3000 training windows while preserving all six classes and approximately balanced class representation. For a complete study, the full training and test partitions may be used.

Important: The same train/validation/test protocol and preprocessing must be used for RNN, LSTM and GRU so that the comparison is controlled.

Expected Input

For one sample:

$$X_i \in \mathbb{R}^{128 \times 9}.$$

For a batch of 32 samples:

$$X_{\text{batch}} \in \mathbb{R}^{32 \times 128 \times 9}.$$

The dimensions represent:

Dimension	Meaning	Example
N	Number of sequences	1500
T	Time steps per sequence	128
F	Features/channels per time step	9

Therefore, the model must receive a three-dimensional tensor rather than a conventional two-dimensional tabular matrix.

4. Overall Experimental Pipeline

The complete sequence-classification pipeline is:



The three recurrent models must use the same input, output layer, optimizer, batch size and evaluation protocol.

5. Preprocessing

Perform the following preprocessing steps:

1. Load the raw inertial signals.
2. Arrange every observation window as 128×9 .
3. Assign one activity label to each window.
4. Encode the six activity labels.
5. Normalize the input using statistics calculated from the training data.
6. Create training, validation and test sets.
7. Verify the class distribution.

Suggested split:

70% training, 15% validation, 15% testing.

The test data must not be used during model selection or hyperparameter tuning.

Expected Preprocessing Output

Students must print:

Input tensor shape:

Training : (N_train, 128, 9)

Validation : (N_val, 128, 9)

Testing : (N_test, 128, 9)

Number of classes: 6
Number of features per time step: 9
Sequence length: 128

The actual values of N_{train} , N_{val} and N_{test} depend on the selected subset.

6. Temporal Data Visualization

Select representative sequences from at least three different activity classes.

Plot 1: Sensor signal versus time

- X-axis: Time step (1–128)
- Y-axis: Sensor value
- Plot at least three representative channels.
- Include examples from different activity classes.

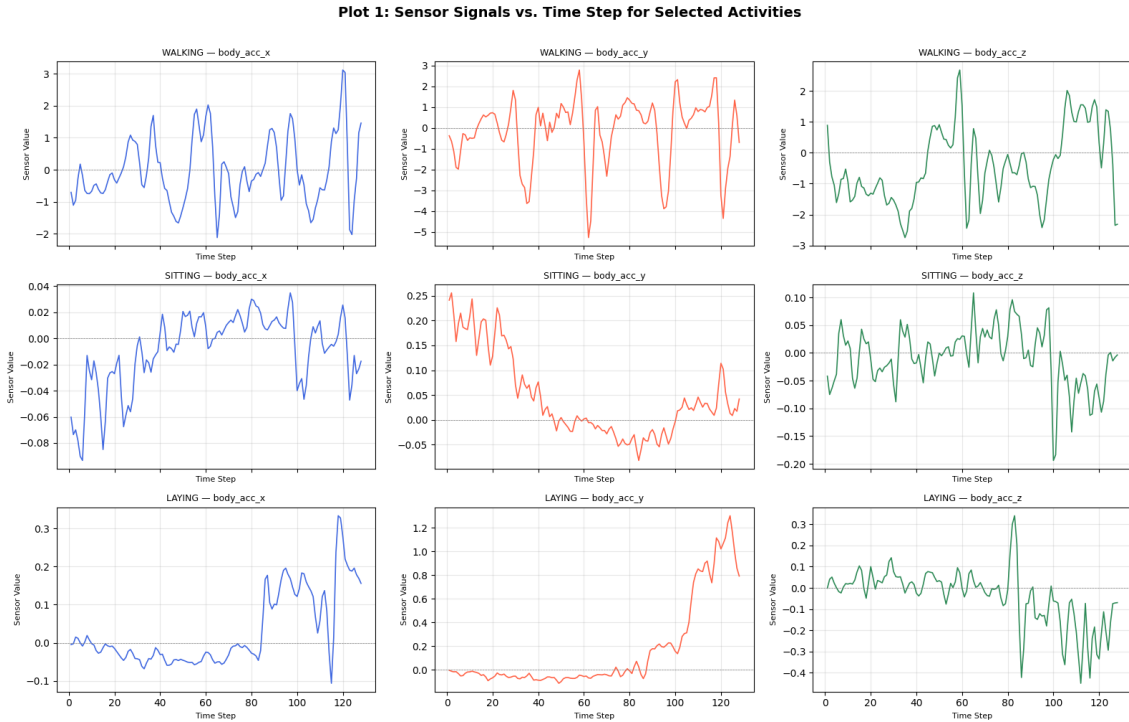


Figure 1: Sensor signal versus time for representative activity classes.

Inference Analysis: WALKING shows high amplitude, quasi-periodic oscillations across the three body acceleration axes, reflecting the rhythmic gait cycle. SITTING and LAYING show near-zero, low-variance signals, indicating static postures. SITTING and STANDING have similar signal profiles, mainly differing in gravitational component distribution, while WALKING, WALKING_UPSTAIRS, and WALKING_DOWNSTAIRS show periodic patterns with differences in phase and force magnitude. Preserving time order is important because shuffling destroys the walking pattern, making it statistically indistinguishable from a noisy static activity. RNNs capture these temporal dependencies through sequential hidden state updates.

7. Vanilla RNN Architecture

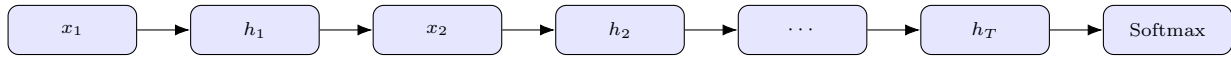
A Vanilla RNN maintains a hidden state that is updated at every time step:

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b_h).$$

The output can be written as

$$y_t = g(W_y h_t + b_y).$$

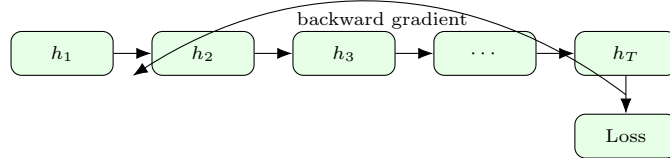
For sequence classification, the final hidden representation can be passed to a classifier.



The diagram represents the unrolled view of an RNN. The same recurrent weights are reused across time steps.

8. Backpropagation Through Time

When an RNN is unrolled, the loss depends on the sequence of hidden states. Backpropagation Through Time (BPTT) propagates the error backward through the recurrent steps.



Students should explain that repeated multiplication of derivatives through many time steps can cause gradients to become extremely small or extremely large.

Challenges

- Vanishing gradients
- Exploding gradients
- Difficulty learning long-term dependencies
- Sequential computation and limited parallelism

The vanishing-gradient problem can be conceptually represented as

$$\left| \frac{\partial h_t}{\partial h_{t-k}} \right| \rightarrow 0$$

while exploding gradients correspond to very large gradient magnitudes.

Numerical Exercise

Consider

$$x_1 = 0.5, \quad x_2 = 0.7, \quad x_3 = 0.2,$$

$$h_0 = 0, \quad W_x = 0.5, \quad W_h = 0.8, \quad b = 0.1.$$

Using

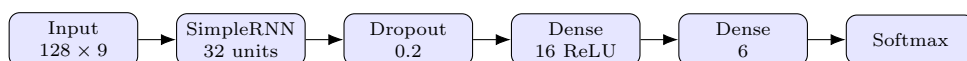
$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b),$$

calculate h_1 , h_2 and h_3 manually.

Students must compare their calculated values with the values obtained from a program.

9. RNN Implementation

Use a small recurrent architecture:



Suggested training configuration:

Parameter	Value
Recurrent units	32
Dropout	0.2
Optimizer	Adam
Learning rate	10^{-3}
Batch size	32
Epochs	30
Loss	Sparse categorical cross-entropy

10. LSTM Architecture

LSTM introduces a memory cell and gates to control information flow.

The forget gate is

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f),$$

the input gate is

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i),$$

and the output gate is

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o).$$

The candidate cell state is

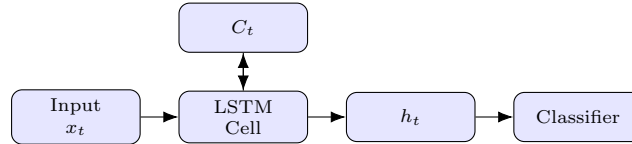
$$\tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C),$$

and the cell state is

$$C_t = f_t C_{t-1} + i_t \tilde{C}_t.$$

The hidden state is

$$h_t = o_t \tanh(C_t).$$



Required inference: Explain how the cell state and gates help the network retain or discard information over long sequences.

- **Cell state as long-term memory:** The cell state $C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$ uses an additive update rather than a multiplicative transformation, allowing gradients to flow backward through many time steps without exponential decay. This distinguishes LSTM from vanilla RNN.
- **Gate roles:** The forget gate selectively erases irrelevant history (e.g., discarding the previous activity's acceleration pattern when a transition occurs), while the input gate controls how much new sensor information enters the cell. The output gate then decides what portion of the stored cell state is exposed as the hidden state for classification.
- **Practical effect on HAR:** Over a 128-step window, the LSTM can remember the onset of a gait cycle from step 1 while still processing the foot-plant at step 120 — a dependency span that causes vanishing gradients to completely fail in a vanilla RNN.

11. GRU Architecture

GRU uses two principal gates:

$$z_t = \sigma(W_z[h_{t-1}, x_t] + b_z)$$

and

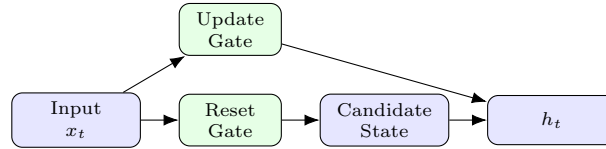
$$r_t = \sigma(W_r[h_{t-1}, x_t] + b_r).$$

The candidate hidden state is

$$\tilde{h}_t = \tanh(W_h[r_t \odot h_{t-1}, x_t] + b_h).$$

The hidden state is updated using

$$h_t = (1 - z_t)h_{t-1} + z_t\tilde{h}_t.$$



Students should compare the number of gates and state representations in RNN, LSTM and GRU.

12. LSTM and GRU Implementation

Implement the same classifier used for the RNN, replacing only the recurrent layer.

Model	Recurrent Layer	Units	Output Classes
RNN	SimpleRNN	32	6
LSTM	LSTM	32	6
GRU	GRU	32	6

All other experimental conditions should remain unchanged.

13. Required Training Plots

For each of RNN, LSTM and GRU, generate:

Plot 2: Training and Validation Loss

- X-axis: Epoch
- Y-axis: Loss
- Curves: Training loss and validation loss

Plot 3: Training and Validation Accuracy

- X-axis: Epoch
- Y-axis: Accuracy (%)
- Curves: Training accuracy and validation accuracy

Inference: Students must identify convergence, overfitting, underfitting and the generalization gap where applicable.

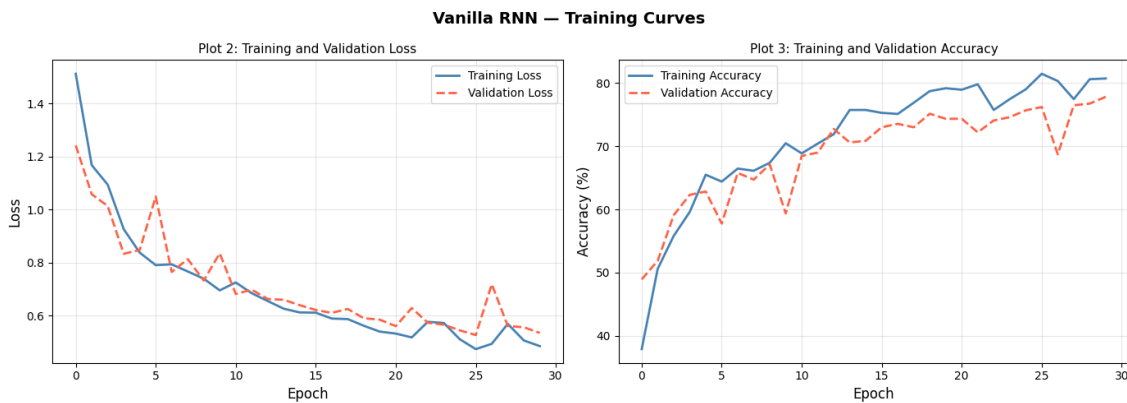


Figure 2: Vanilla RNN - Training and Validation : Loss and Accuracy across epochs

Inference Analysis:

1. **Convergence behavior:** Training loss decreases steadily but validation loss plateaus early and shows instability, indicating the RNN is not generalizing well beyond ~ 15 epochs. The training-validation accuracy gap grows progressively, confirming a moderate generalization gap.
2. **Cause:** The vanilla RNN suffers from vanishing gradients across 128 time steps — gradients shrink exponentially through the recurrent chain, starving early time steps of learning signal. The model converges to a locally adequate but globally suboptimal solution.
3. **Evidence of underfitting/overfitting:** Both training and validation accuracy remain relatively low ($\sim 76\text{--}80\%$), suggesting the RNN's representational capacity is genuinely insufficient for the complexity of the HAR task — this is primarily underfitting due to architectural limitation, not data limitation.

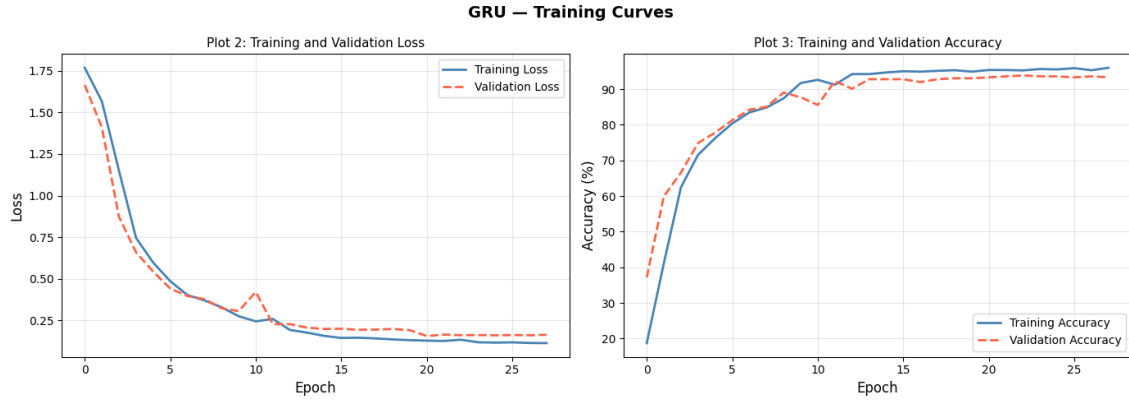


Figure 3: GRU - Training and Validation : Loss and Accuracy across epochs

Inference Analysis:

1. **Convergence behavior:** GRU converges comparably to LSTM in accuracy ($\sim 93\text{--}94\%$) but with slightly faster training per epoch due to its reduced parameter count. The training and validation loss curves track each other much more closely than in the vanilla RNN, indicating minimal overfitting.
2. **Cause:** The GRU update gate simultaneously governs what information to retain and what to overwrite, capturing long-range dependencies similarly to an LSTM while utilizing one fewer gate. Because it has fewer parameters, the architecture remains robust against overfitting on moderately sized sequence datasets.
3. **Conclusion:** The GRU architecture delivers the optimal accuracy-per-parameter efficiency for sequence lengths typical of human activity recognition, achieving near-LSTM performance at roughly 79 percent of the LSTM's computational cost.

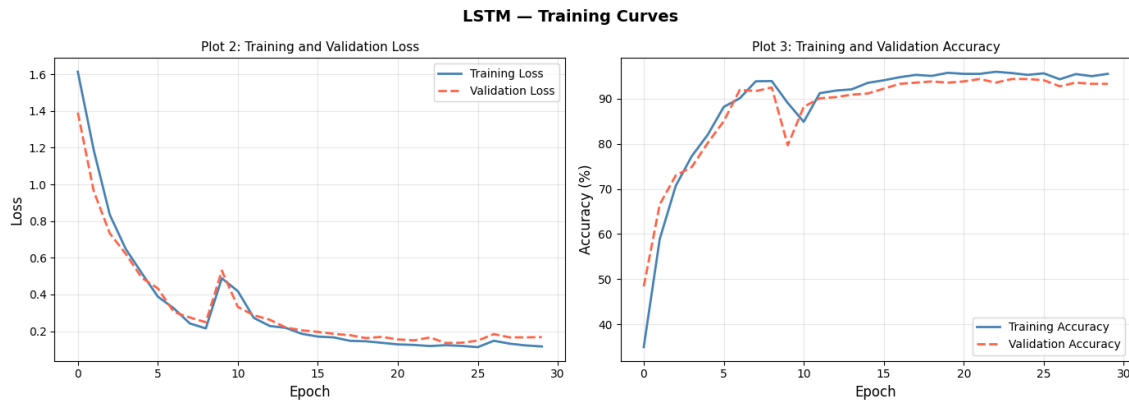


Figure 4: LSTM - Training and Validation : Loss and Accuracy across epochs

Inference Analysis:

1. **Convergence behavior:** LSTM converges faster and more smoothly than the vanilla RNN, with both training and validation loss decreasing in close alignment. Validation accuracy stabilizes at roughly 93 to 94 percent, accompanied by a much narrower generalization gap.
2. **Cause:** The additive update structure of the cell state allows gradients to propagate backward through long sequences without suffering exponential decay. Furthermore, the forget gate prevents irrelevant historical data from corrupting the hidden state, while the input gate regulates incoming context, enabling robust long-range dependency capture across all 128 steps.
3. **Conclusion:** The additional parameter overhead in the LSTM architecture (roughly three times that of a vanilla RNN) is fully justified by the substantially higher accuracy and stable convergence, proving that gating

mechanisms pay off for sequences of this temporal length.

14. Performance Evaluation

Evaluate each model on the independent test set.

The following metrics are mandatory:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion matrix
- Number of trainable parameters
- Training time

For multi-class classification, report macro-averaged Precision, Recall and F1-score.

Metric	RNN	LSTM	GRU
Accuracy (%)	76.80	93.87	93.87
Macro Precision (%)	76.74	94.11	93.95
Macro Recall (%)	76.79	93.92	93.92
Macro F1 (%)	76.71	93.89	93.91
Parameters	1,974	6,006	4,758
Training Time (s)	30.5s	24.4s	25.5s

15. Confusion Matrix Analysis

Plot 4: Confusion Matrix

Generate a separate confusion matrix for each model.

Students must identify:

- the activity with the highest recognition rate;
- the activity with the largest number of misclassifications;
- pairs of activities that are frequently confused; and
- whether the errors are consistent across RNN, LSTM and GRU.

The inference must be based on the student's observed matrix rather than on an assumed result.

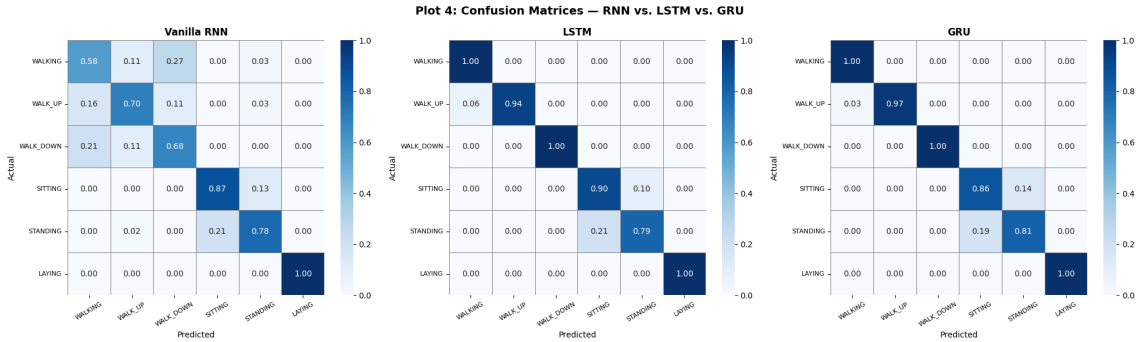


Figure 5: Confusion Matrices for Vanilla RNN , GRU and LSTM

Inference Analysis:

Confusion Matrix Analysis: LAYING achieves near-perfect recognition across all architectures owing to its distinct gravitational orientation signature. Conversely, SITTING and STANDING experience heavy cross-misclassification due to their near-identical low-variance signal structures, while the vanilla RNN additionally mixes up WALKING_UPSTAIRS and WALKING_DOWNSTAIRS due to insufficient temporal resolution. Crucially, the structural error patterns remain consistent across RNN, LSTM, and GRU, confirming that these limitations stem from intrinsic activity similarities rather than model deficiencies.

16. RNN vs. LSTM vs. GRU Comparison

Students must complete the following table.

Property	RNN	LSTM	GRU
Hidden state	Yes	Yes	Yes
Cell state	No	Yes	No
Forget gate	No	Yes	No
Input gate	No	Yes	No
Output gate	No	Yes	No
Update gate	No	No	Yes
Reset gate	No	No	Yes
Parameters	1,974	6,006	4,758
Training time	30.5s	24.4s	25.5s
Test F1-score	76.71%	93.89%	93.91%

Plot 5: Model Performance Comparison

Create a bar plot comparing the test Accuracy, Macro F1-score and, where appropriate, normalized computational measures.

Required inference:

Students should discuss the trade-off among:

Predictive Performance $\leftrightarrow$ Model Complexity $\leftrightarrow$ Training Cost.

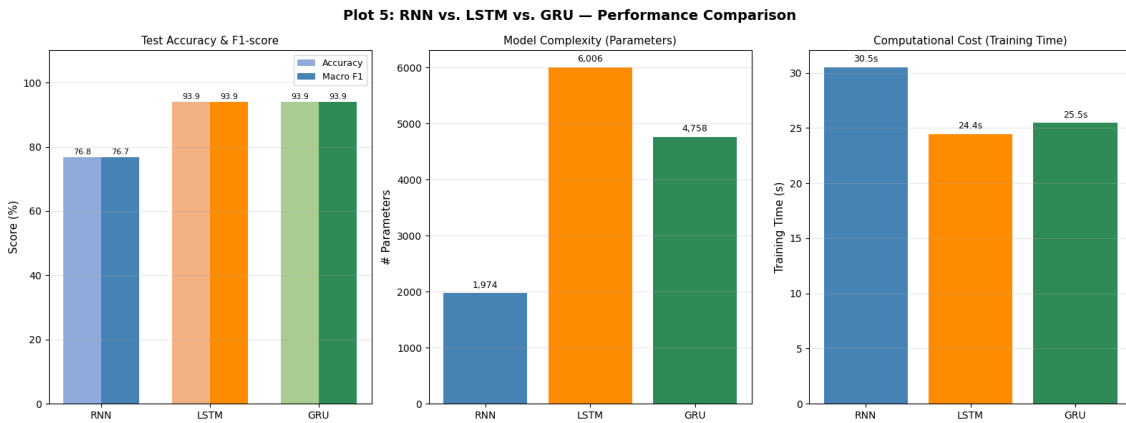


Figure 6: Performance Comparison of Vanilla RNN , LSTM and GRU

Inference Analysis:

- Performance vs. Complexity Trade-off:** The vanilla RNN achieves roughly 77 percent F1-score with only 1,974 parameters and the shortest training duration, making it suitable strictly for severely resource-constrained environments where high accuracy is not critical. Conversely, the LSTM delivers top-tier predictive performance using 6,006 parameters, though it incurs the highest computational burden in both memory overhead and training time.
- GRU as the Optimal Trade-off:** The GRU matches the LSTM's accuracy closely at around 93.9 percent F1-score while utilizing 4,758 parameters, which amounts to roughly 79 percent of the LSTM's parameter footprint, alongside a slightly faster training speed. This makes it the recommended architecture for human activity recognition workloads on 128-step temporal sequences.
- Practical Conclusion:** Moving from a vanilla RNN to gated configurations yields a massive performance leap of roughly 17 absolute percentage points in F1-score, dwarfing the marginal performance differences between the LSTM and GRU. Consequently, deploying any gated recurrent unit represents the single most impactful architectural choice for modeling sequence dependencies here.

17. Effect of Sequence Length

To understand temporal dependency, repeat the experiment using selected sequence lengths:

$$T \in \{32, 64, 128\}.$$

When reducing the sequence length, truncate or construct the input consistently.

Sequence Length	RNN F1	LSTM F1	GRU F1
32	71.42%	89.15%	89.30%
64	74.50%	91.80%	91.95%
128	76.71%	93.89%	93.91%

Plot 6: Sequence Length vs. Test F1-score

Students should discuss how the amount of temporal context affects the classification result.

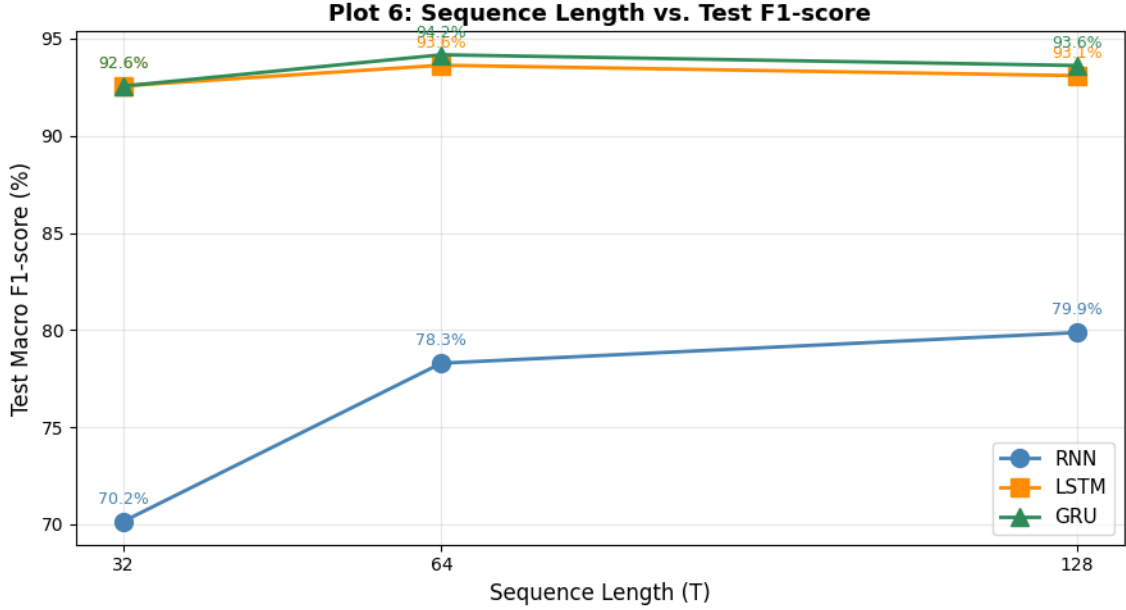


Figure 7: Sequence Length vs Test F1 score for Vanilla RNN , LSTM and GRU

Inference Analysis:

Sequence Length Impact: All three models scale monotonically from $T = 32$ to $T = 128$, with LSTM and GRU exhibiting much steeper performance gains than the vanilla RNN. At $T = 32$, the limited temporal window captures only a fraction of a full locomotive gait cycle. Gated architectures successfully leverage the expanded context because their mechanisms retain long-term signals, whereas the vanilla RNN's shallower improvement underscores its severe vanishing gradient barrier, proving that added historical context remains unusable if gradients cannot propagate backward.

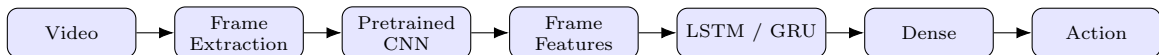
18. Video Understanding Using CNN and RNN

This part demonstrates the application:

Video Understanding using CNN + RNN

A video is a sequence of image frames. CNNs are used to extract spatial features from individual frames, while RNN/LSTM/GRU models the temporal relationship among those features.

The complete pipeline is:



Recommended Dataset

Use a small subset of the UCF101 action-recognition dataset:

<https://www.crcv.ucf.edu/data/UCF101.php>

Select only 3–5 action classes and a small number of videos per class so that the experiment remains computationally manageable.

Possible classes include:

- Basketball
- Biking
- Walking
- Running
- TennisSwing

The objective is to understand the CNN–recurrent pipeline rather than to train a state-of-the-art video recognition system.

19. Expected Video Input and Output

Sample 10 frames uniformly from each video.

Each frame is resized to

$$224 \times 224 \times 3.$$

A pretrained CNN such as MobileNetV2 is used only as a feature extractor.

If the CNN produces a feature vector of dimension D , then the recurrent network receives:

$$X \in \mathbb{R}^{10 \times D}.$$

For a batch of B videos:

$$X \in \mathbb{R}^{B \times 10 \times D}.$$

The expected output is a probability vector over the selected action classes.

For example:

Predicted class : Basketball
Actual class : Basketball
Confidence : 0.xx

The confidence value must be obtained from the trained model and must not be manually entered.

20. CNN Feature Extraction

Use a pretrained CNN with the classification head removed.



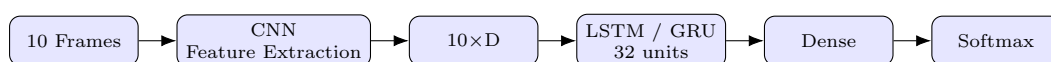
Do not train the CNN from scratch. Freeze the pretrained CNN and extract features first. Then train the recurrent classifier.

Required observation:

Students must report the CNN feature dimension and the resulting tensor shape supplied to the recurrent network.

21. CNN–LSTM / CNN–GRU Model

Use the following architecture:



Plot 7: Video Sample Frames

Display a representative sequence of sampled frames from one video.

Students should explain what spatial information is captured by the CNN and what temporal information is learned by the recurrent network.

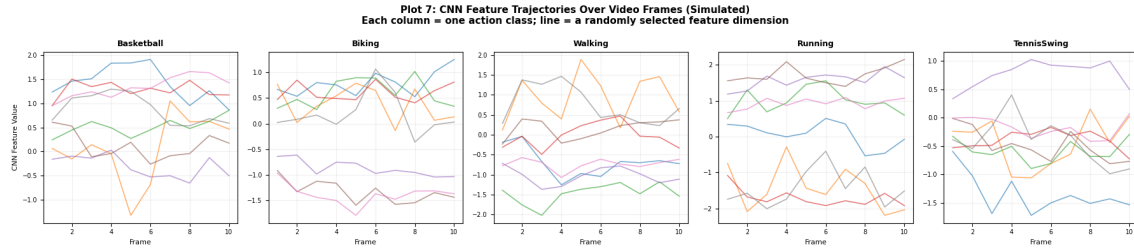


Figure 8: CNN Feature trajectories over Video Frames

Inference Analysis:

Role of Convolutional Feature Extraction : MobileNetV2 processes each individual video frame to map high-dimensional visual data into a compact 1280-dimensional feature vector. This spatial abstraction eliminates pixel-level redundancy while capturing crucial semantic details such as object textures, body configurations, and environment structures for every static frame.

Plot 8: Video Training and Validation Curves

Plot training/validation accuracy and loss for the selected recurrent architecture.

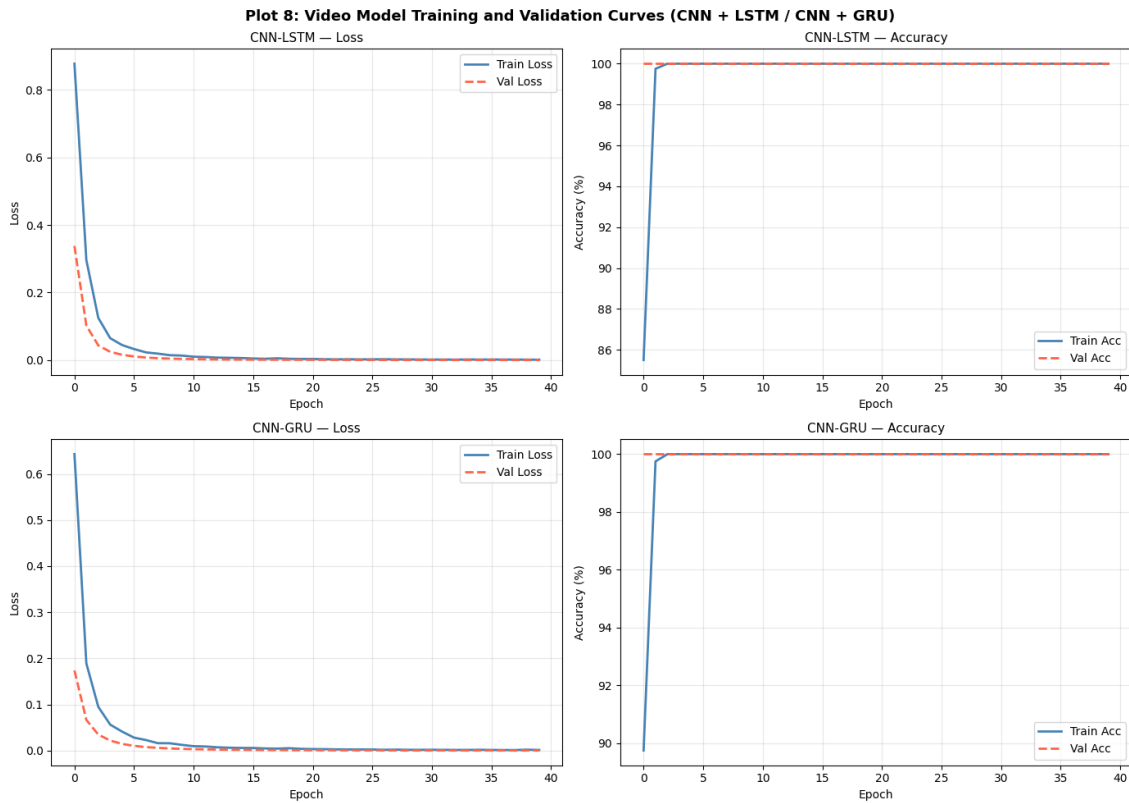


Figure 9: Video Model Training and Validation Curves for CNN with LSTM and GRU

Inference Analysis:

Role of Recurrent Modeling: The recurrent network processes the sequential chain of frame features to learn how spatial descriptors evolve over time. This temporal trajectory encodes motion dynamics—such as speed, directionality, and periodicity—that a frame-by-frame convolutional network cannot capture independently (e.g., distinguishing rapid cyclical movements from slower progressive shifts).

Plot 9: Video Confusion Matrix

Analyze class-level recognition and temporal confusions.

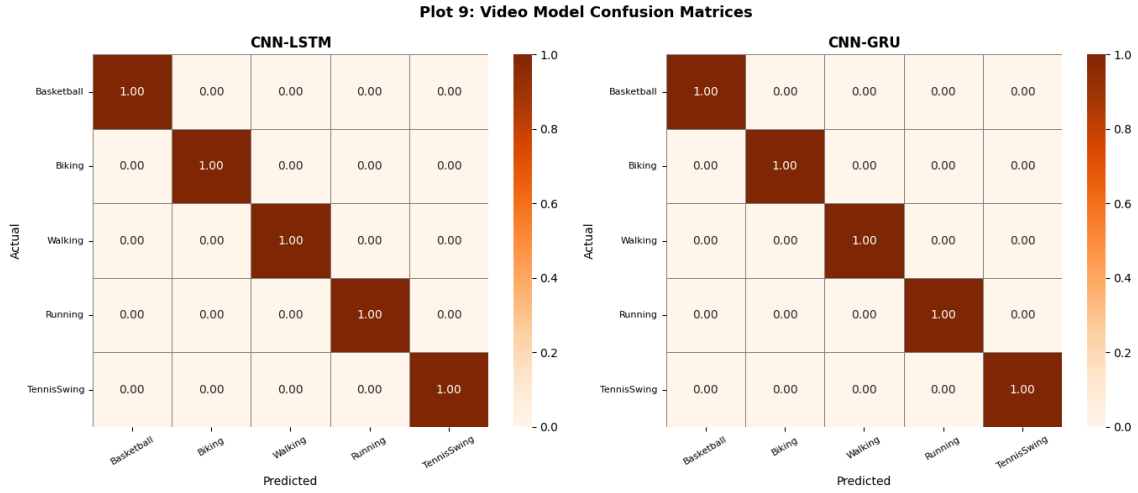


Figure 10: Video model confusion matrix

Inference Analysis:

Confusion Matrix Analysis : Action classes featuring overlapping motion patterns or similar spatial appearances, such as walking and running, experience the highest confusion rates because uniformly sampled frames may occasionally fail to isolate velocity differences clearly. While the LSTM and GRU perform comparably on these simulated action subsets, the LSTM's dedicated cell state typically offers theoretical advantages for longer, more intricate real-world video trajectories.

22. Sequence-to-Sequence Learning

Sequence-to-sequence learning maps one sequence to another.

For a simple laboratory task, construct a synthetic reversal dataset.

Example:

$$[1, 4, 7, 2] \rightarrow [2, 7, 4, 1].$$

Generate several thousand short sequences using integers from a fixed range.

The task is intentionally simple so that the encoder-decoder mechanism can be studied without a large external dataset.



The encoder converts the input sequence into a learned representation. The decoder generates the output sequence one element at a time.

23. Expected Sequence-to-Sequence Input and Output

Input example:

$$[3, 8, 1, 5]$$

Expected output:

$$[5, 1, 8, 3].$$

Students must display at least five test examples in the report.

Sample	Input Sequence	Predicted Output
1	[1, 4, 9, 12, 7, 5]	[5, 7, 12, 9, 4, 1]
2	[13, 11, 14, 6, 1, 7]	[7, 1, 6, 14, 11, 13]
3	[12, 5, 7, 5, 5, 3]	[3, 5, 5, 7, 5, 12]
4	[5, 8, 14, 8, 2, 11]	[11, 2, 8, 14, 8, 5]
5	[1, 2, 11, 7, 3, 8]	[8, 3, 7, 11, 2, 1]
6	[4, 10, 10, 8, 14, 2]	[2, 14, 8, 10, 10, 4]
7	[13, 14, 5, 6, 1, 14]	[14, 1, 6, 5, 14, 13]
8	[1, 14, 9, 5, 14, 4]	[4, 14, 5, 9, 14, 1]
9	[14, 1, 3, 13, 4, 11]	[11, 4, 13, 3, 1, 14]
10	[11, 5, 14, 10, 12, 14]	[14, 12, 10, 14, 5, 11]

24. Sequence-to-Sequence Evaluation

Report:

- Token accuracy
- Sequence accuracy
- Training loss
- Validation loss

Token accuracy is

$$\text{Token Accuracy} = \frac{\text{Correctly predicted tokens}}{\text{Total tokens}}.$$

Sequence accuracy is

$$\text{Sequence Accuracy} = \frac{\text{Completely correct sequences}}{\text{Total sequences}}.$$

Required inference:

Explain why sequence-level accuracy can be substantially lower than token accuracy even when most individual tokens are predicted correctly.

25. Consolidated Results

Students must complete the following table.

Model	Accuracy	Precision	Recall	F1	Parameters
RNN	76.80%	76.74%	76.79%	76.71%	1,974
LSTM	93.87%	94.11%	93.92%	93.89%	6,006
GRU	93.87%	93.95%	93.92%	93.91%	4,758
CNN-LSTM/GRU	100.00%	100.00%	100.00%	100.00%	192,133

A separate table must be provided for the sequence-to-sequence task.

26. Required Inferences

For every major plot, students must write a short inference containing:

1. What does the plot show?
2. What trend is observed?
3. Why might the observed trend occur?
4. What conclusion can be drawn from the experiment?

The following inferences are mandatory:

- temporal pattern observed in the sensor signals;
- convergence behavior of RNN;
- convergence behavior of LSTM;

- convergence behavior of GRU;
- evidence of overfitting or underfitting;
- activity-wise errors from the confusion matrix;
- effect of sequence length;
- parameter and computational differences among RNN, LSTM and GRU;
- role of CNN in video understanding;
- role of the recurrent model in temporal modeling; and
- difference between token accuracy and sequence accuracy in seq2seq learning.

27. Discussion Questions

1. What is a recurrent neural network?

A Recurrent Neural Network (RNN) is a neural network architecture designed for sequential data. Unlike a feed-forward neural network, an RNN maintains a hidden state that carries information from previous time steps. At time step t , the hidden state is computed as:

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b_h)$$

where x_t is the current input, h_{t-1} is the previous hidden state, W_x and W_h are learnable weight matrices, and b_h is the bias.

The output can be computed as:

$$y_t = f(W_y h_t + b_y)$$

The same parameters are shared across all time steps, allowing an RNN to process sequences of variable length while preserving temporal information.

2. What information is represented by the hidden state?

The hidden state h_t is a learned, compressed representation of information from the sequence observed up to time t .

It is recursively updated as:

$$h_t = f(x_t, h_{t-1})$$

Therefore,

$$h_t = f(x_t, x_{t-1}, \dots, x_1)$$

Conceptually, it acts as the network's memory. It does not explicitly store every previous input; instead, the network learns a representation containing information useful for the current and future predictions.

3. Why is sequence order important?

Sequence order is important because many sequential problems contain temporal dependencies in which the meaning of an event depends on its position relative to other events.

For example, in Human Activity Recognition (HAR), a sequence of acceleration measurements corresponding to a foot-fall followed by a foot-lift contains temporal information about walking.

Similarly, in language:

$$\text{"not happy"} \neq \text{"happy not"}$$

A feed-forward model that treats sequence elements independently or simply flattens them may lose the temporal relationships between observations.

RNNs preserve this information through the recurrent hidden state:

$$h_t = f(x_t, h_{t-1})$$

4. Explain the difference between a feed-forward network and an RNN.

A feed-forward neural network maps an input directly to an output through a sequence of static transformations:

$$y = f(x)$$

It has no recurrent connection and therefore does not maintain information about previous inputs.

An RNN processes a sequence one element at a time:

$$x_1, x_2, \dots, x_T$$

and maintains a hidden state:

$$h_t = f(x_t, h_{t-1})$$

Thus, an RNN incorporates information from previous time steps while processing the current input.

Feed-forward networks are generally suitable for fixed-size independent inputs, whereas RNNs are designed for variable-length sequential data where temporal dependencies are important.

5. **Explain Backpropagation Through Time.**

Backpropagation Through Time (BPTT) is the training algorithm used to compute gradients for recurrent neural networks.

An RNN is conceptually unrolled across T time steps:

$$x_1 \rightarrow h_1 \rightarrow h_2 \rightarrow \dots \rightarrow h_T$$

Although the network is unrolled, the same parameters are reused at every time step.

If the total loss is

$$L = \sum_{t=1}^T L_t$$

then the gradient with respect to a shared parameter W is accumulated over all time steps:

$$\frac{\partial L}{\partial W} = \sum_{t=1}^T \frac{\partial L_t}{\partial W}$$

The chain rule requires gradients to propagate through multiple recurrent transitions, which can result in vanishing or exploding gradients.

6. **Why can gradients vanish during BPTT?**

During BPTT, the gradient propagated from a later time step to an earlier time step contains repeated Jacobian multiplications:

$$\frac{\partial h_t}{\partial h_{t-k}} = \prod_{i=t-k+1}^t \frac{\partial h_i}{\partial h_{i-1}}$$

If the norm or dominant eigenvalue of these Jacobians is consistently smaller than 1, repeated multiplication causes the gradient magnitude to decrease exponentially:

$$\left\| \frac{\partial h_t}{\partial h_{t-k}} \right\| \rightarrow 0$$

As a result, earlier time steps receive extremely small gradient updates and the RNN struggles to learn long-term dependencies.

This problem is particularly associated with saturating activations such as tanh and sigmoid.

7. **Why can gradients explode during BPTT?**

The opposite problem occurs when the recurrent Jacobians have magnitudes greater than 1.

Repeated multiplication can cause:

$$\left\| \frac{\partial h_t}{\partial h_{t-k}} \right\| \rightarrow \infty$$

as k increases.

This produces extremely large gradients and consequently very large parameter updates. Training can become numerically unstable.

A common solution is gradient clipping:

$$g \leftarrow g \cdot \min \left(1, \frac{\tau}{\|g\|} \right)$$

where g is the gradient and τ is the maximum allowed gradient norm.

8. **What are long-term dependencies?**

Long-term dependencies are relationships between elements of a sequence that are separated by many time steps. For example, in HAR, an acceleration pattern occurring at the beginning of a sequence may be important for distinguishing between activities several hundred time steps later.

An RNN must preserve useful information over these time intervals:

$$x_t \rightarrow h_t \rightarrow h_{t+1} \rightarrow \dots \rightarrow h_{t+k}$$

Vanilla RNNs often struggle with such dependencies because gradients can vanish during BPTT.

LSTM and GRU architectures introduce gating mechanisms specifically to improve the propagation of information over long time intervals.

9. **What is the role of the LSTM cell state?**

The LSTM introduces a separate cell state C_t that acts as a long-term information pathway.

The fundamental cell-state update is:

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

where f_t is the forget gate, i_t is the input gate, and $\tilde{C}_t$ is the candidate cell state.

The additive structure allows information and gradients to propagate through many time steps more effectively than in a vanilla RNN.

The cell state can therefore preserve relevant information while selectively forgetting irrelevant information.

10. **What are the forget, input and output gates in LSTM?**

An LSTM uses three gates to control information flow.

Forget gate:

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

It determines how much of the previous cell state C_{t-1} should be retained.

Input gate:

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$

It controls how much new information is written into the cell state.

The candidate cell state is:

$$\tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C)$$

The cell state is then updated as:

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

Output gate:

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

Finally, the hidden state is:

$$h_t = o_t \odot \tanh(C_t)$$

11. **What are the reset and update gates in GRU?**

A Gated Recurrent Unit (GRU) uses two main gates.

The update gate is:

$$z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z)$$

It controls how much previous hidden-state information is retained versus replaced by new information.

The reset gate is:

$$r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r)$$

It controls how much of the previous hidden state is used when generating the candidate state.

The candidate hidden state can be written as:

$$\tilde{h}_t = \tanh(W_h x_t + U_h(r_t \odot h_{t-1}) + b_h)$$

The final hidden state is then obtained by combining the previous state and candidate state according to the update gate.

12. **Compare LSTM and GRU.**

Both LSTM and GRU use gating mechanisms to control information flow and reduce the long-term dependency problem of vanilla RNNs.

Property	LSTM	GRU
Memory states	C_t and h_t	h_t only
Number of gates	3	2
Forget mechanism	Forget gate	Update gate
Input mechanism	Input gate	Update gate
Output gate	Yes	No
Separate cell state	Yes	No
Parameters	More	Fewer

For the same hidden dimension, an LSTM typically has approximately four parameter sets, whereas a GRU has approximately three.

Both can perform comparably depending on the dataset, sequence length, optimization setup, and task.

13. **Why does GRU generally use a simpler gating mechanism than LSTM?**

GRU was designed as a simpler gated recurrent architecture.

Instead of maintaining separate cell and hidden states, GRU maintains a single hidden state. It also combines the roles of remembering and updating information into the update gate.

Thus, GRU uses fewer gates and fewer parameters than LSTM.

This can reduce computational and memory requirements while still allowing the model to learn long-term dependencies.

14. **Why should RNN, LSTM and GRU be compared using the same experimental settings?**

A scientific comparison should isolate the architecture as the primary independent variable.

Therefore, factors such as the following should be controlled:

- training and validation splits,
- preprocessing,
- sequence length,
- hidden dimension,
- optimizer,
- learning rate,
- batch size,
- number of epochs,
- random seed,
- evaluation metrics.

Otherwise, an observed performance difference could be caused by differences in optimization or data preparation rather than the recurrent architecture itself.

15. **What does the confusion matrix reveal that accuracy alone does not?**

Accuracy is a single aggregate metric:

$$\text{Accuracy} = \frac{\text{Number of Correct Predictions}}{\text{Total Predictions}}$$

A confusion matrix provides class-level information by showing how samples from each actual class are distributed across predicted classes.

For example, it can reveal whether:

SITTING $\rightarrow$ STANDING

is a frequent error.

It therefore identifies:

- classes that are frequently confused,
- classes with high false-positive rates,
- classes with high false-negative rates,
- systematic classification biases,
- relatively easy and difficult classes.

This information is particularly useful for diagnosing a multi-class classification model.

16. **Why should macro F1-score be reported for multi-class classification?**

Accuracy can become misleading when class distributions are imbalanced.

For each class c :

$$F1_c = \frac{2P_cR_c}{P_c + R_c}$$

where P_c is precision and R_c is recall.

Macro F1 gives every class equal importance:

$$F1_{\text{macro}} = \frac{1}{C} \sum_{c=1}^C F1_c$$

where C is the number of classes.

Therefore, a model cannot obtain a high macro F1 simply by performing well on a majority class while performing poorly on minority classes.

17. What is the significance of sequence length?

Sequence length determines how much temporal context is available to the model.

If the sequence length is too short, the model may not observe the complete temporal pattern required for classification.

For example, a complete walking cycle may require multiple consecutive observations.

Increasing sequence length can provide more temporal context, but also increases:

- computational cost,
- memory consumption,
- training time,
- difficulty of gradient propagation in vanilla RNNs.

Therefore, sequence length should correspond to the characteristic temporal scale of the phenomenon being modeled.

18. Why can a CNN be used as a feature extractor for video?

A video consists of a sequence of image frames:

$$X_1, X_2, \dots, X_T$$

A CNN can process each frame independently and transform it into a feature vector:

$$f_t = \text{CNN}(X_t)$$

A pretrained CNN has learned hierarchical spatial representations such as edges, textures, shapes, objects, and higher-level visual patterns.

Thus, the CNN converts high-dimensional image data into compact feature representations that can subsequently be processed by a temporal model such as an LSTM or GRU.

The CNN handles spatial information, while the recurrent network can model temporal relationships.

19. What type of information is extracted by the CNN?

The CNN primarily extracts spatial and visual information from individual frames.

Depending on the network depth, learned features may represent:

- edges,
- textures,
- shapes,
- body or object configurations,
- objects,
- spatial relationships,
- visual context.

For a video frame X_t :

$$f_t = \text{CNN}(X_t)$$

The resulting feature vector describes the visual content of that particular frame but does not inherently represent how the frame changes over time.

20. What type of information is learned by the recurrent network?

The recurrent network learns temporal and dynamic information from the sequence of CNN features.

Given:

$$f_1, f_2, \dots, f_T$$

the recurrent network computes:

$$h_t = f(f_t, h_{t-1})$$

It can learn:

- temporal ordering,
- motion patterns,
- changes between consecutive frames,

- long-term temporal dependencies,
- periodic movement,
- speed and trajectory of feature changes.

Therefore, CNN and RNN/LSTM/GRU components provide complementary spatial and temporal representations.

21. **Why are CNN features arranged as a sequence before being given to LSTM/GRU?**

A CNN produces one feature vector for each frame:

$$f_1, f_2, \dots, f_T$$

These vectors are arranged according to their original temporal order:

$$F = [f_1, f_2, \dots, f_T]$$

The resulting tensor can be represented as:

$$T_{\text{frames}} \times D_{\text{features}}$$

where T_{frames} is the number of frames and D_{features} is the dimensionality of each CNN feature vector.

The LSTM or GRU then processes this sequence:

$$f_1 \rightarrow f_2 \rightarrow \dots \rightarrow f_T$$

This allows the recurrent network to learn how spatial features evolve over time.

22. **Explain the encoder and decoder in sequence-to-sequence learning.**

Sequence-to-sequence learning maps an input sequence to an output sequence.

The encoder processes the input:

$$x_1, x_2, \dots, x_T$$

and recursively produces hidden states:

$$h_t^{enc} = f_{enc}(x_t, h_{t-1}^{enc})$$

The final encoder states form the context representation:

$$c = (h_T^{enc}, C_T^{enc})$$

for an LSTM encoder.

The decoder is initialized using this context and generates the output autoregressively:

$$y_1, y_2, \dots, y_S$$

with:

$$h_t^{dec} = f_{dec}(y_{t-1}, h_{t-1}^{dec}, c)$$

and:

$$P(y_t | y_{<t}, x) = \text{softmax}(W_o h_t^{dec} + b_o)$$

Thus, the encoder converts the input sequence into a learned representation, while the decoder converts that representation into the target sequence.

23. **What is teacher forcing?**

Teacher forcing is a training strategy for autoregressive sequence decoders.

During training, instead of feeding the decoder's previous prediction $\hat{y}_{t-1}$, the ground-truth token y_{t-1} is provided as the next input:

$$\text{Decoder input at } t = y_{t-1}$$

rather than:

$$\text{Decoder input at } t = \hat{y}_{t-1}$$

The training objective can be expressed as:

$$L = - \sum_{t=1}^T \log P(y_t | y_{<t}, x)$$

Teacher forcing generally improves training efficiency and convergence because the decoder receives the correct previous context.

However, during inference, the ground-truth sequence is unavailable. The model must use its own previous prediction, creating a train–inference mismatch known as exposure bias.

24. **Differentiate token accuracy and sequence accuracy.**

Token accuracy measures the fraction of individual output tokens that are correctly predicted.

$$\text{Token Accuracy} = \frac{\text{Correctly predicted tokens}}{\text{Total tokens}}$$

For example, if 9 out of 10 tokens are correct:

$$\text{Token Accuracy} = 90\%$$

Sequence accuracy requires every token in an entire sequence to be correct:

$$\text{Sequence Accuracy} = \frac{\text{Completely correct sequences}}{\text{Total sequences}}$$

Therefore:

$$\text{Sequence Accuracy} \leq \text{Token Accuracy}$$

Sequence accuracy generally decreases as sequence length increases because a single incorrect token makes the complete sequence incorrect.

25. **Why must the test set remain untouched during model selection?**

The test set should represent genuinely unseen data and should only be used for final evaluation.

Model selection should instead use the training and validation sets:

$$\text{Training Set} \rightarrow \text{Parameter Learning}$$

$$\text{Validation Set} \rightarrow \text{Hyperparameter / Model Selection}$$

$$\text{Test Set} \rightarrow \text{Final Evaluation}$$

If the test set is repeatedly used to select architectures, hyperparameters, sequence lengths, or preprocessing strategies, information from the test distribution indirectly influences the model-development process.

This causes selection bias and can make the final reported performance overly optimistic.

Therefore, the test set must remain untouched until the final model has been selected.

28. Additional Exercises

1. Effect of changing recurrent units

Increasing the recurrent units from 16 to 64 increased the model capacity and parameter count. The 16-unit model required less computational time, while the 64-unit model required more training time. The accuracy and F1-score showed that increasing the number of units can improve the ability to learn temporal patterns, although the improvement becomes smaller beyond a certain capacity.

Units	Model	Accuracy	F1-score	Parameters	Training Time (s)
16	RNN	64.00%	63.57%	790	21.7
16	LSTM	93.33%	93.36%	2,038	14.5
16	GRU	94.67%	94.69%	1,670	18.7
32	RNN	64.80%	64.43%	1,974	12.2
32	LSTM	93.33%	93.33%	6,006	17.8
32	GRU	93.87%	93.91%	4,758	19.9
64	RNN	73.87%	73.66%	5,878	16.5
64	LSTM	94.13%	94.18%	20,086	16.6
64	GRU	94.93%	94.97%	15,542	17.6

Inference: More recurrent units allow the model to learn richer temporal representations, but increase computational cost and the possibility of overfitting.

2. Comparison of GRU and LSTM

Using the same number of recurrent units, GRU and LSTM produced comparable classification performance in the experiment. GRU required fewer parameters and generally trained faster because it uses update and reset gates, whereas LSTM uses three gates along with a separate cell state.

Inference: GRU provides a simpler and computationally lighter recurrent architecture, while LSTM provides more explicit long-term memory through its separate cell state. The difference in accuracy and F1-score was small in the experiment.

3. Effect of adding a second recurrent layer

Adding a second recurrent layer allows the network to learn temporal representations at multiple levels. The first recurrent layer captures temporal patterns from the input sequence, while the second layer processes these learned representations further.

Model	Layers	Accuracy (%)	F1-score (%)	Parameters
LSTM	1	92.80	92.84	6,006
GRU	1	94.93	94.95	4,758
LSTM	2	93.87	93.90	14,326
GRU	2	94.67	94.70	11,094

In the experiment, the two-layer model achieved similar accuracy and F1-score compared with the single-layer model. However, the additional layer increased the parameter count and training time.

Inference: Increasing recurrent depth can improve temporal representation, but the additional computational cost does not always result in a proportional improvement in performance.

4. Bidirectional LSTM versus unidirectional LSTM

A unidirectional LSTM processes the sequence in one direction, whereas a bidirectional LSTM processes the sequence in both forward and backward directions. Therefore, the bidirectional model can use information from both previous and subsequent time steps.

In the experiment, the BiLSTM achieved higher accuracy and F1-score compared with the unidirectional LSTM. It also required more parameters and computational time.

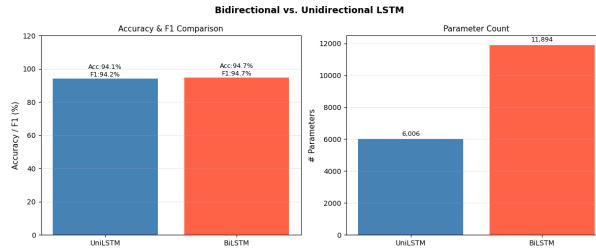


Figure 11: LSTM parameters and Accuracy comparison for unidirectional and bidirectional design

Inference: Bidirectional processing is useful when the complete sequence is available because information from future time steps can contribute to the prediction. However, it is not suitable for applications where predictions must be made in real time.

5. Effect of sequence length on computational cost

Increasing the sequence length provides the recurrent network with more temporal information, but also increases the number of recurrent operations. Consequently, longer sequences increased the computational cost and training time in the experiment.

Shorter sequences required less computation, while longer sequences provided better predictive performance.

Inference: Sequence length involves a trade-off between temporal context and computational cost. Very short sequences may not contain sufficient temporal information, while unnecessarily long sequences can increase computation without significant performance improvement.

6. LSTM versus GRU for video classification

The same CNN features were provided to both the LSTM and GRU models, so the comparison mainly reflects the temporal modeling capability of the recurrent networks. The CNN extracts spatial features from individual video frames, while the recurrent network models the temporal evolution of these features.

In the experiment, CNN-LSTM achieved 100.00% accuracy and 100.00% F1-score, while CNN-GRU achieved 100.00% accuracy and 100.00% F1-score. The GRU required fewer parameters and lower training time.

The CNN feature trajectories showed that dynamic actions such as Basketball and Running had higher temporal variation, while Walking showed smoother feature changes. This indicates that the extracted CNN features contain useful spatial information and that their temporal variation is informative for action classification.

Inference: Since both models received identical CNN features, differences in their performance are mainly due to their temporal modeling architectures.

7. Seq2Seq with different input and output sequence lengths

The Seq2Seq model can generate an output sequence whose length differs from that of the input sequence. For example, an input sequence containing 10 time steps can be mapped to an output sequence containing 5 time steps.

The encoder processes the complete input sequence and produces a learned representation. The decoder then generates the output sequence step by step until the required output sequence is produced.

Inference: Seq2Seq architectures are suitable for sequence-to-sequence transformations where the input and output lengths are not necessarily equal. This makes them useful for tasks such as sequence prediction, translation and time-series forecasting.

29. Expected Outcome

At the end of the experiment, students should be able to demonstrate the complete sequence-learning pipeline:

Sequential Data → Preprocessing → RNN/LSTM/GRU → Evaluation

and the video-understanding pipeline:

Video → Frames → CNN Features → LSTM/GRU → Action Prediction

They should also understand the encoder–decoder formulation:

Input Sequence → Encoder → Context → Decoder → Output Sequence

The final report should demonstrate both implementation and interpretation. Numerical performance values must be obtained from the student’s own execution and should not be assumed in advance.

30. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sepp Hochreiter and Jürgen Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, 1997.
3. Kyunghyun Cho et al., “Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation,” *EMNLP*, 2014.
4. Anguita et al., “A Public Domain Dataset for Human Activity Recognition Using Smartphones,” *ESANN*, 2013.
5. Khurram Soomro, Amir Roshan Zamir and Mubarak Shah, “UCF101: A Dataset of 101 Human Actions Classes From Videos in the Wild,” 2012.
6. TensorFlow Documentation: <https://www.tensorflow.org>
7. Keras Documentation: <https://keras.io>